

DATA STRUCTURES HOMEWORK PROGRAMS EX-2

1) Define an integer array. Declare an integer pointer and initialize it to point to the first element of the array. Use pointer arithmetic to traverse the array and print the elements. Modify the elements of the array using pointer arithmetic.

```
#include <stdio.h>

int main()
{
    int n;

    printf("Enter the size of the array: ");
    scanf("%d", &n);

    int arr[n];

    int *ptr;

    printf("Enter %d elements:\n", n);
    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    ptr = arr;

    printf("\nOriginal array elements:\n");
    for (int i = 0; i < n; i++)
    {
        printf("%d ", *(ptr + i));
    }

    for (int i = 0; i < n; i++)
    {
        *(ptr + i) = *(ptr + i) + 5;
    }

    printf("\n\nArray elements after modification:\n");
    for (int i = 0; i < n; i++)
    {
        printf("%d ", *(ptr + i));
    }
}
```

```
    return 0;
}
```

Output

Enter the size of the array: 5

Enter 5 elements:

10

20

30

40

50

Original array elements:

10 20 30 40 50

Array elements after modification:

15 25 35 45 55

2) Calculate string length using pointers.

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    char str[100];
```

```
    char *ptr;
```

```
    int length = 0;
```

```
    printf("Enter a string: ");
```

```
    fgets(str, sizeof(str), stdin);
```

```
    ptr = str;
```

```
    while (*ptr != '\0')
```

```
    {
```

```
        if (*ptr == '\n')
```

```
            break;
```

```
        length++;
```

```
        ptr++;
```

```

    }

    printf("Length of the string = %d", length);

    return 0;
}

```

Output

Enter a string: Hello World

Length of the string = 11

3) Write a function that calculates both the sum and product of two numbers. Return these two results using pointers passed as arguments to the function.

```

#include <stdio.h>

void calculate(int a, int b, int *sum, int *product)
{
    *sum = a + b;
    *product = a * b;
}

int main()
{
    int a, b;
    int sum, product;
    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);
    calculate(a, b, &sum, &product);
    printf("Sum = %d\n", sum);
    printf("Product = %d\n", product);
    return 0;
}

```

Output

Enter two numbers: 10 5

Sum = 15

Product = 50

4) Create an array of structures. Use dynamic memory allocation to allocate memory space and use a structure pointer with pointer arithmetic to iterate and access/modify members.

```
#include <stdio.h>

#include <stdlib.h>

struct Student
{
    int rollno;
    char name[50];
    float marks;
};

int main()
{
    int n;
    printf("Enter the number of students: ");
    scanf("%d", &n);
    struct Student *ptr;
    ptr = (struct Student *)malloc(n * sizeof(struct Student));
    if (ptr == NULL)
    {
        printf("Memory allocation failed");
        return 1;
    }
    for (int i = 0; i < n; i++)
    {
        printf("\nEnter details of student %d:\n", i + 1);
        printf("Roll number: ");
        scanf("%d", &(ptr + i)->rollno);
        printf("Name: ");
        scanf("%s", (ptr + i)->name);
        printf("Marks: ");
        scanf("%f", &(ptr + i)->marks);
    }
}
```

```

printf("\nStudent details:\n");
for (int i = 0; i < n; i++)
{
    printf("\nRoll Number: %d", (ptr + i)->rollno);
    printf("\nName: %s", (ptr + i)->name);
    printf("\nMarks: %.2f\n", (ptr + i)->marks);
}
for (int i = 0; i < n; i++)
{
    (ptr + i)->marks += 5;
}
printf("\nAfter modifying marks:\n");
for (int i = 0; i < n; i++)
{
    printf("\nRoll Number: %d", (ptr + i)->rollno);
    printf("\nName: %s", (ptr + i)->name);
    printf("\nMarks: %.2f\n", (ptr + i)->marks);
}
free(ptr);
return 0;
}

```

Output

Enter the number of students: 2

Enter details of student 1:

Roll number: 101

Name: Arun

Marks: 80

Enter details of student 2:

Roll number: 102

Name: Ravi

Marks: 75

Student details:

Roll Number: 101

Name: Arun

Marks: 80.00

Roll Number: 102

Name: Ravi

Marks: 75.00

After modifying marks:

Roll Number: 101

Name: Arun

Marks: 85.00

Roll Number: 102

Name: Ravi

Marks: 80.00

5) Define two functions with same signature. Declare a function pointer and use the pointer to call both functions.

```
#include <stdio.h>
```

```
int square(int n)
```

```
{  
    return n * n;
```

```
}
```

```
int cube(int n)
```

```
{  
    return n * n * n;
```

```
}
```

```
int main()
```

```
{  
    int n;  
    int (*ptr)(int);  
    printf("Enter a number: ");  
    scanf("%d", &n);
```

```

    ptr = square;

    printf("Square = %d\n", ptr(n));

    ptr = cube;

    printf("Cube = %d\n", ptr(n));

    return 0;
}

```

Output

Enter a number: 5

Square = 25

Cube = 125

6) Declare an integer, a pointer to it and a double pointer. Print the values and addresses at each level.

```

#include <stdio.h>

int main()
{
    int n;

    int *ptr;

    int **dptr;

    printf("Enter an integer: ");

    scanf("%d", &n);

    ptr = &n;

    dptr = &ptr;

    printf("\nValue of n = %d", n);

    printf("\nAddress of n = %p", (void *)&n);

    printf("\n\nValue of ptr = %p", (void *)ptr);

    printf("\nValue pointed by ptr = %d", *ptr);

    printf("\nAddress of ptr = %p", (void *)&ptr);

    printf("\n\nValue of dptr = %p", (void *)dptr);

    printf("\nValue pointed by dptr = %p", (void *)*dptr);

    printf("\nValue pointed by double dereference = %d", **dptr);

    printf("\nAddress of dptr = %p", (void *)&dptr);
}

```

```
    return 0;
}
```

Output

Enter an integer: 10

Value of n = 10

Address of n = 0x7ffe119ce9ec

Value of ptr = 0x7ffe119ce9ec

Value pointed by ptr = 10

Address of ptr = 0x7ffe119ce9e0

Value of dptr = 0x7ffe119ce9e0

Value pointed by dptr = 0x7ffe119ce9ec

Value pointed by double dereference = 10

Address of dptr = 0x7ffe119ce9d8

=== Code Execution Successful ===

7) Sort an array of floating point numbers using Selection Sort.

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n;
```

```
    float arr[100], temp;
```

```
    printf("Enter the size of the array: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter %d floating point numbers:\n", n);
```

```
    for (int i = 0; i < n; i++)
```

```
    {
```

```
        scanf("%f", &arr[i]);
```

```
    }
```

```
    for (int i = 0; i < n - 1; i++)
```

```
    {
```

```
        int min = i;
```

```

        for (int j = i + 1; j < n; j++)
        {
            if (arr[j] < arr[min])
            {
                min = j;
            }
        }
        temp = arr[i];
        arr[i] = arr[min];
        arr[min] = temp;
    }
    printf("\nSorted array:\n");
    for (int i = 0; i < n; i++)
    {
        printf("%.2f ", arr[i]);
    }
    return 0;
}

```

Output

Enter the size of the array: 5

Enter 5 floating point numbers:

4.5

2.3

9.8

1.2

6.7

Sorted array:

1.20 2.30 4.50 6.70 9.80

8) Define a structure Student with id, name and grade, create an array of Student structures and sort them based on different fields.

```

#include <stdio.h>

#include <string.h>

struct Student
{
    int id;
    char name[50];
    float grade;
};

int main()
{
    int n, choice;
    struct Student s[100], temp;
    printf("Enter the number of students: ");
    scanf("%d", &n);
    for (int i = 0; i < n; i++)
    {
        printf("\nEnter details of student %d:\n", i + 1);
        printf("ID: ");
        scanf("%d", &s[i].id);
        printf("Name: ");
        scanf("%s", s[i].name);
        printf("Grade: ");
        scanf("%f", &s[i].grade);
    }
    printf("\nSort by:\n");
    printf("1. ID\n");
    printf("2. Name\n");
    printf("3. Grade\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);
    for (int i = 0; i < n - 1; i++)

```

```

{
    for (int j = i + 1; j < n; j++)
    {
        int condition = 0;
        if (choice == 1 && s[i].id > s[j].id)
            condition = 1;
        if (choice == 2 && strcmp(s[i].name, s[j].name) > 0)
            condition = 1;
        if (choice == 3 && s[i].grade > s[j].grade)
            condition = 1;
        if (condition)
        {
            temp = s[i];
            s[i] = s[j];
            s[j] = temp;
        }
    }
}

printf("\nSorted Student Details:\n");
for (int i = 0; i < n; i++)
{
    printf("ID: %d, Name: %s, Grade: %.2f\n",
        s[i].id, s[i].name, s[i].grade);
}
return 0;
}

```

Output

Enter the number of students: 3

Enter details of student 1:

ID: 103

Name: Ravi

Grade: 85.5

Enter details of student 2:

ID: 101

Name: Arun

Grade: 92.0

Enter details of student 3:

ID: 102

Name: Bala

Grade: 78.5

Sort by:

1. ID

2. Name

3. Grade

Enter your choice: 1

Sorted Student Details:

ID: 101, Name: Arun, Grade: 92.00

ID: 102, Name: Bala, Grade: 78.50

ID: 103, Name: Ravi, Grade: 85.50

9) Sort a set of negative numbers using Radix Sort.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int getMax(int arr[], int n)
```

```
{
```

```
    int max = arr[0];
```

```

for (int i = 1; i < n; i++)
{
    if (arr[i] > max)
        max = arr[i];
}
return max;
}

```

```

void radixSort(int arr[], int n)
{
    int max = getMax(arr, n);
    for (int exp = 1; max / exp > 0; exp *= 10)
    {
        int output[n];
        int count[10] = {0};
        for (int i = 0; i < n; i++)
            count[(arr[i] / exp) % 10]++;
        for (int i = 1; i < 10; i++)
            count[i] += count[i - 1];
        for (int i = n - 1; i >= 0; i--)
        {
            output[count[(arr[i] / exp) % 10] - 1] = arr[i];
            count[(arr[i] / exp) % 10]--;
        }
        for (int i = 0; i < n; i++)
            arr[i] = output[i];
    }
}

int main()
{
    int n;

```

```

printf("Enter the size of the array: ");
scanf("%d", &n);
int arr[n];
int positive[n], negative[n];
int p = 0, q = 0;
printf("Enter %d numbers:\n", n);
for (int i = 0; i < n; i++)
{
    scanf("%d", &arr[i]);
    if (arr[i] < 0)
        negative[q++] = -arr[i];
    else
        positive[p++] = arr[i];
}
radixSort(negative, q);
radixSort(positive, p);
printf("\nSorted array:\n");
for (int i = q - 1; i >= 0; i--)
    printf("%d ", -negative[i]);
for (int i = 0; i < p; i++)
    printf("%d ", positive[i]);
return 0;
}

```

Output

Enter the size of the array: 7

Enter 7 numbers:

-45

23

-12

56

-89

10

-34

Sorted array:

-89 -45 -34 -12 10 23 56

10. Write a C program to copy one array to another using pointers.

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n;
```

```
    printf("Enter the size of the array: ");
```

```
    scanf("%d", &n);
```

```
    int arr1[n], arr2[n];
```

```
    int *p1, *p2;
```

```
    printf("Enter %d elements:\n", n);
```

```
    for (int i = 0; i < n; i++)
```

```
    {
```

```
        scanf("%d", &arr1[i]);
```

```
    }
```

```
    p1 = arr1;
```

```
    p2 = arr2;
```

```
    for (int i = 0; i < n; i++)
```

```
    {
```

```
        *(p2 + i) = *(p1 + i);
```

```
    }
```

```
    printf("\nFirst array:\n");
```

```
    for (int i = 0; i < n; i++)
```

```
    {
```

```
        printf("%d ", *(p1 + i));
```

```
    }
```

```
    printf("\n\nSecond array after copying:\n");
```

```
    for (int i = 0; i < n; i++)
```

```
{  
    printf("%d ", *(p2 + i));  
}  
return 0;  
}
```

Output

Enter the size of the array: 5

Enter 5 elements:

10

20

30

40

50

First array:

10 20 30 40 50

Second array after copying:

10 20 30 40 50